

LA JERARQUÍA — de quién extiende qué

uvm_void	
└ uvm_object	crear, copiar, comparar
└ uvm_transaction	
└ uvm_sequence_item	tu transaction
└ uvm_sequence	tus sequences
└ uvm_report_object	
└ uvm_component	está en el árbol: nombre jerárquico, padre y fases
└ uvm_test	tus tests
└ uvm_env	tu env
└ uvm_agent	tu agent
└ uvm_driver	tu driver
└ uvm_monitor	tus monitores
└ uvm_scoreboard	tu scoreboard
└ uvm_subscriber	tu cobertura
└ uvm_sequencer	tal cual, con typedef

La línea que importa: un object se crea, se usa y se tira; un component vive toda la simulación. Por eso el constructor de uno es (name) y el del otro (name, parent).

LAS NUEVE FASES — sólo una consume tiempo

Fase	Para qué	Orden
build	instanciar los componentes	top-down
connect	conectar los ports	bottom-up
end_of_elaboration	jerarquía lista, antes de simular	bottom-up
start_of_simulation	último aviso antes del tiempo 0	bottom-up
run	task: acá pasa la simulación	un thread c/u
extract	juntar los datos de la corrida	bottom-up
check	decidir si pasó o no	bottom-up
report	imprimir el veredicto	bottom-up
final	cerrar archivos y salir	bottom-up

build es la única top-down porque el padre tiene que existir para crear al hijo; las demás necesitan lo contrario. Todas son function salvo run, y por eso ninguna otra puede esperar un flanco.

Y el que decide cuándo termina no es run: son las objections. raise al empezar, drop al terminar, y un drain_time si queda algo en vuelo sin comparar.

EL HANDSHAKE — sequence ↔ driver

La sequence	El driver
task body(); req = tipo::type_id ::create("req"); start_item(req); // espera assert(req.randomize()); finish_item(req); // espera // req.result ya volvió endtask	task run_phase(uvm_phase f); forever begin seq_item_port .get_next_item(cmd); bfm.send_op(cmd.A, cmd.B, cmd.op); seq_item_port .item_done(); end endtask

Es un préstamo, no una copia: la sequence sigue teniendo el handle, así que el driver le escribe el resultado **adentro del item** antes de item_done(). Sin ese item_done() la sequence se cuelga en su finish_item() **sin un solo mensaje de error**.

LAS SIETE PERILLAS DE DEBUG — cinco son plusargs: no se recompila nada

Perilla	Para qué	El síntoma que resuelve
VLT_TRACE=1	volcar las ondas para GTKWave	cuando el log ya no alcanza — el 47 % del trabajo
+UVM_VERBOSEITY=UVM_HIGH	ver los mensajes de los monitores	el scoreboard grita en todas : el monitor muestrea mal, el DUT casi nunca es
+UVM OBJECTION_TRACE	quién la levantó y quién la bajó	termina en t = 0 y dice PASS, o no termina nunca
+UVM_TIMEOUT=5ms	un techo, en vez de esperar	la simulación se colgó y no sabés dónde
+UVM_CONFIG_DB_TRACE	quién puso qué, y quién lo leyó	el config_db «no encuentra»: mirá el ámbito del set, no el get
uvm_root::get() .print_topology()	el árbol que UVM armó de verdad	el árbol no es el que dibujaste: un create() sin factory, o un override tardío
set_report_severity_ action_hier(..., UVM_COUNT)	matar la corrida al N-ésimo error	el log de cuatro gigas cuando el scoreboard falla en cada transacción

Regla de oro: el sospechoso número uno nunca es el DUT — es el testbench que lo mira. La cobertura da 0 %: falta el new() o el sample() del covergroup. Todo esto corre sin licencias: Verilator + UVM 2020.3.1.